
Traineeship Application

Sprint Report

Sakellariou Ioanna, AM: 5125

Mouselimi Amalia-Georgia, AM: 5074

Zioga Despoina, AM: 5055

VERSIONS HISTORY

Date	Version	Description	Author
20.05.2025	v.1.0.0	A non-profitable traineeship app	Ioanna Sakellariou, Amalia-Georgia Mouselimi, Despoina Zioga

1 Introduction

1.1 Purpose

The goal of this project is to create an application designed to assist the University's traineeship committee in overseeing and managing both available and assigned traineeship opportunities. The system will enable companies to post open traineeship positions, while students will have the ability to browse and explore these opportunities. The traineeship committee will handle the assignment of positions to students based on various selection criteria. Additionally, the committee will designate professors to supervise the assigned traineeships. Both professors and company representatives will be responsible for conducting the final evaluations of the students' performance during their traineeship.

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes out Scrum team and specifies this Sprint's backlog. Section 3 specifies the main design concepts for this release of the project.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	Sakellariou Ioanna / Mouselimi Amalia-Georgia / Zioga Despoina
Scrum Master	Sakellariou Ioanna
Development Team	Sakellariou Ioanna / Mouselimi Amalia-Georgia / Zioga Despoina

2.2 Sprints

Sprint No	Begin Date	End Date	Number of weeks	User stories
1	09.03.2025	26.03.2025	2.5	US1, US2, US3 (login-logout)
2	29.03.2025	5.04.2025	1	US4, US7, US13, US16
3	6.04.2025	27.04.2025	3	US5, US8, US9, US10, US11, US14

4	30.04.2025	14.05.2025	2	US17, US18, US19 US20, US6
5	15.05.2025	20.05.2025	1	US12, US15, US21

3 Use Cases

3.1 Create Account

Use case ID	UC1
Actors	User
Pre conditions	The system has loaded.
Main flow of events	1. The use case starts when the user presses "Register". 2. The system displays user information fields: Username, Full Name, Role (Student, Company, Professor, Committee). 3. The user fills in the form and presses the 'register' button. 4. The system confirms successful registration.
Alternative flow 1	At any time the user may leave the application.
Post conditions	The account is saved in the system.

3.2 Login

Use case ID	UC2
Actors	User
Pre conditions	User has an account.
Main flow of events	1. The use case starts when the user presses "login". 2. The user enters the credentials. 3. The system authenticates and logs the user in.
Alternative flow 1	At any time the user may leave the application.

Alternative flow 2	Invalid credentials, it shows an error message.
Post conditions	User is logged in.

3.3 Logout

Use case ID	UC3
Actors	User
Pre conditions	User is logged in.
Main flow of events	1. The use case starts when the user presses "Logout" 2. System ends session and redirects to login.
Alternative flow 1	At any time the user may leave the application.
Post conditions	User is logged out.

3.4 Edit Student Profile

Use case ID	UC4
Actors	Student
Pre conditions	Logged in as a student.
Main flow of events	1. The use case starts when the user presses "Edit Profile". 2. Enters Full Name, University ID, Interests, Skills, Preferred Location. 3. Submits the form.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the edit form by pressing "Back to Dashboard".
Post conditions	Student profile is saved.

3.5 Edit Company Profile

Use case ID	UC5
Actors	Company
Pre conditions	Logged in as a company.
Main flow of events	1. The use case starts when the user presses "Edit Profile". 2. Enters Company Name and Location. 3. Submits the form.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the edit form by pressing "Back to Dashboard".
Post conditions	Company profile is saved.

3.6 Edit Professor Profile

Use case ID	UC6
Actors	Professor
Pre conditions	Logged in as a professor.
Main flow of events	1. The use case starts when the user presses "Edit Profile". 2. Enters Full Name and Interests. 3. Submits the form.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the edit form by pressing "Back to Dashboard".
Post conditions	Professor profile is saved.

3.7 Apply For Traineeship

Use case ID	UC7
Actors	Student
Pre conditions	Student is logged in and has a profile.
Main flow of events	1. The use case starts when the student clicks “Available Positions”. 2. Views the available traineeships. 3. Select a position and clicks “Apply”. 4. The System confirms the application.
Alternative flow 1	At any time the user may leave the application.
Post conditions	The application is stored and visible to the committee.

3.8 Fill in Traineeship Logbook

Use case ID	UC8
Actors	Student
Pre conditions	Traineeship is assigned.
Main flow of events	1. The use case starts when the user presses “Logbook”. 2. Enters their report . 3. Submits log entry.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the edit form by pressing “Back to Dashboard”.
Post conditions	Entry is saved.

3.9 Announce Traineeship Position

Use case ID	UC9
Actors	Company

Pre conditions	Logged in as company and user has pressed “My Posted Positions”.
Main flow of events	1. The use case starts when the user presses “Add Position” 2. Fills in: Start/End Dates, Description, Required Skills and Topics 3. Submits the position. 4. The system saves the position.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the edit form by pressing “Back to Dashboard”.
Post conditions	Position becomes available to students.

3.10 Assign Traineeship to Student

Use case ID	UC10
Actors	Traineeship Committee Member
Pre conditions	Application exists, logged in as committee member and has already pressed “Show Student Applications”.
Main flow of events	1. The use case starts when the user presses “Suggest Positions” on the selected student. 2. Filters available positions based on interests and location. 3. Assign position to the student by pressing “Assign”.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the assign process by pressing “Back to Dashboard”.
Alternative flow 3	At any time the user may leave the assign process by pressing “Back to Applications”.
Post conditions	Traineeship is assigned to student.

3.11 View Supervised Traineeships

Use case ID	UC11
Actors	Professor
Pre conditions	Application exists and professor is logged in.
Main flow of events	1. The use case starts when the user presses “View Supervised Positions”. 2. The system displays a list of traineeships assigned to them. 3. The user can view details for each.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the assign process by pressing “Back to Dashboard”.
Post conditions	The professor sees their assigned traineeships.

3.12 Evaluate a Traineeship and Company

Use case ID	UC12
Actors	Professor
Pre conditions	The professor is logged in and supervises a traineeship.
Main flow of events	1. The use case starts when the user presses “View Supervised Positions”. 2. The system displays the traineeships supervised by the user. 3. The user presses “Evaluate” 4. The system displays an evaluation form. 5. The user rates the student based on motivation, effectiveness and efficiency. 6. The user evaluates the company based on facilities and guidance. 7. The user submits the form by pressing “Submit Evaluation”
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the evaluation process by pressing “Back to Dashboard”.
Post conditions	Evaluation data is saved and viewable by the committee.

3.13 Assign Supervising Professor

Use case ID	UC13
Actors	Committee Member
Pre conditions	Application exists, committee member is logged in and an active traineeship exists.
Main flow of events	1. The use case starts when the user presses “Assign Supervisor”. 2. The user presses “Assign” on the selected Traineeship. 3. The system suggests professors based on interests or workload. 4. The committee assigns a professor by pressing “Assign”. 5. The system updates the record.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the assign process by pressing “Back to Dashboard”.
Post conditions	Professor is assigned to a Traineeship position.

3.14 View Applied Students

Use case ID	UC14
Actors	Committee Member
Pre conditions	Application exists and committee member is logged in.
Main flow of events	1. The use case starts when the user presses “Show Student Applications”. 2. The system loads the student applicants.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the assign process by pressing “Back to Dashboard”.
Post conditions	Committee Member views the applicants and can proceed to match with positions.

3.15 Monitor Traineeship Progress

Use case ID	UC15
Actors	Committee Member
Pre conditions	Application exists, committee member is logged in and traineeships are ongoing.
Main flow of events	1. The use case starts when the user presses “View In-Progress Traineeships”. 2. If no students have applied for a traineeship the user is shown a page that reads “No traineeships in progress” 3. If a student has applied for a traineeship the user is shown a page with the information of the in-progress traineeships.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the monitor process by pressing “Back to Dashboard”.
Post conditions	The committee monitors the traineeships.

3.16 Complete Traineeship with Pass/Fail

Use case ID	UC16
Actors	Committee Member
Pre conditions	Application exists, committee member is logged in, evaluations of professor and company are submitted and user has pressed “View In-Progress Traineeships”.
Main flow of events	1. The use case starts when the user presses “View Evaluations”. 2. The user is redirected to the evaluation review page. 3. The user then can complete a Traineeship with a “pass” or “fail” mark by pressing the corresponding button. 4. The system is updated.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the monitor process by pressing “Back to Dashboard”.
Post conditions	The traineeship is finalized.

3.17 Evaluate a Traineeship

Use case ID	UC17
Actors	Company
Pre conditions	Company is logged in, hosts a traineeship and pressed “Assigned Positions”.
Main flow of events	1. The use case starts when the user presses “Evaluate”. 2. The system displays an evaluation form. 3. The user rates the traineeship based on motivation, effectiveness and efficiency. 4. The user submits the form by pressing “Submit Evaluation”
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the evaluation process by pressing “Back to Dashboard”.
Post conditions	Evaluation data is saved.

3.18 View Available Traineeship Positions

Use case ID	UC18
Actors	Student
Pre conditions	Application exists, student is logged in, a company hosts a traineeship and student has a profile.
Main flow of events	1. The use case starts when the user presses “Available Positions”. 2. The user is shown a page with information about the available positions.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the evaluation process by pressing “Back to Dashboard”.
Post conditions	Student is informed about opportunities.

3.19 Delete Traineeship Position

Use case ID	UC19
Actors	Company
Pre conditions	Application exists, company is logged in, hosts a traineeship and the user is in the tab “Advertised Positions”.
Main flow of events	1. The use case starts when the user presses “Delete”
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the evaluation process by pressing “Back to Dashboard”.
Post conditions	Position is removed

3.20 View Assigned Traineeships

Use case ID	UC20
Actors	Company
Pre conditions	Application exists, company is logged in and a position that the company has announced has been assigned.
Main flow of events	2. The use case starts when the user presses “Assigned Positions” 3. The user is shown informations about the assigned Traineeships.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the evaluation process by pressing “Back to Dashboard”.
Post conditions	Company manages assignments.

3.21 View In-Progress Traineeships

Use case ID	UC21
Actors	Committee Member

Pre conditions	Committee member is logged in, traineeships exist and are active.
Main flow of events	1. The use case starts when the user presses “View”. 2. The user can track status, professor, company.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the evaluation process by pressing “Back to Dashboard”.
Post conditions	Full visibility of running traineeships.

3.22 Search Available Positions for Student

Use case ID	UC22
Actors	Committee Member
Pre conditions	Committee member is logged in, traineeships exist and student profile exist.
Main flow of events	1. The use case starts when the user presses “View”. 2. The user can track status, professor, company.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the evaluation process by pressing “Back to Dashboard”.
Post conditions	Full visibility of running traineeships.

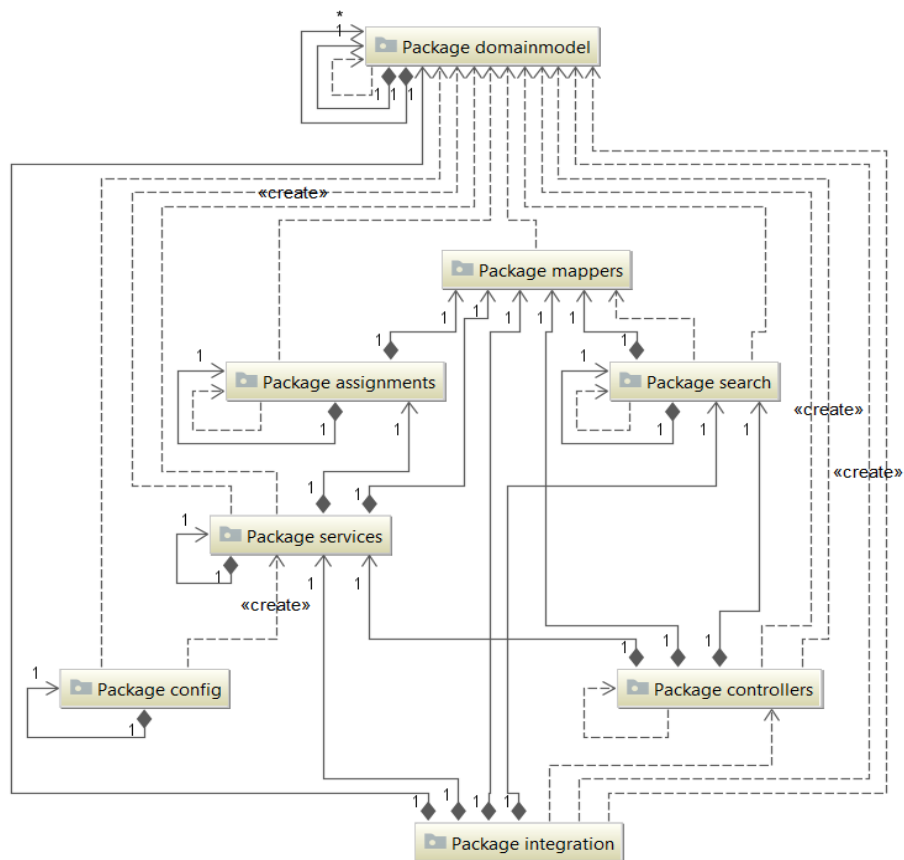
3.23 View Traineeship History

Use case ID	UC23
Actors	Committee Member
Pre conditions	Committee member is logged in, there are completed traineeships stored in the system.
Main flow of events	1. The use case starts when the user presses “View Completed Traineeships”.

	2. The user can see informations about the completed traineeships.
Alternative flow 1	At any time the user may leave the application.
Alternative flow 2	At any time the user may leave the evaluation process by pressing “Back to Dashboard”.
Post conditions	Full visibility of completed traineeships.

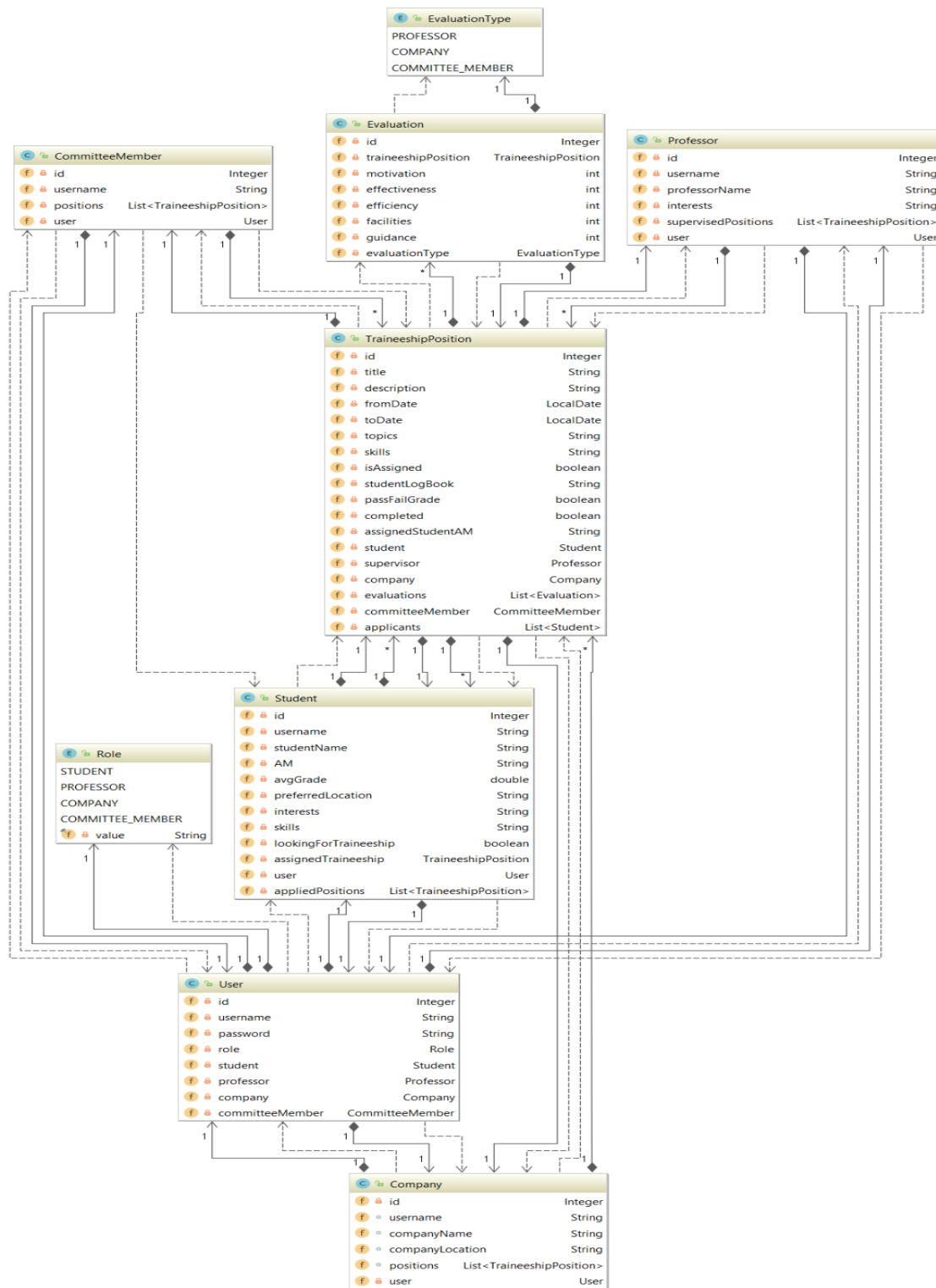
4 Design

4.1 Architecture

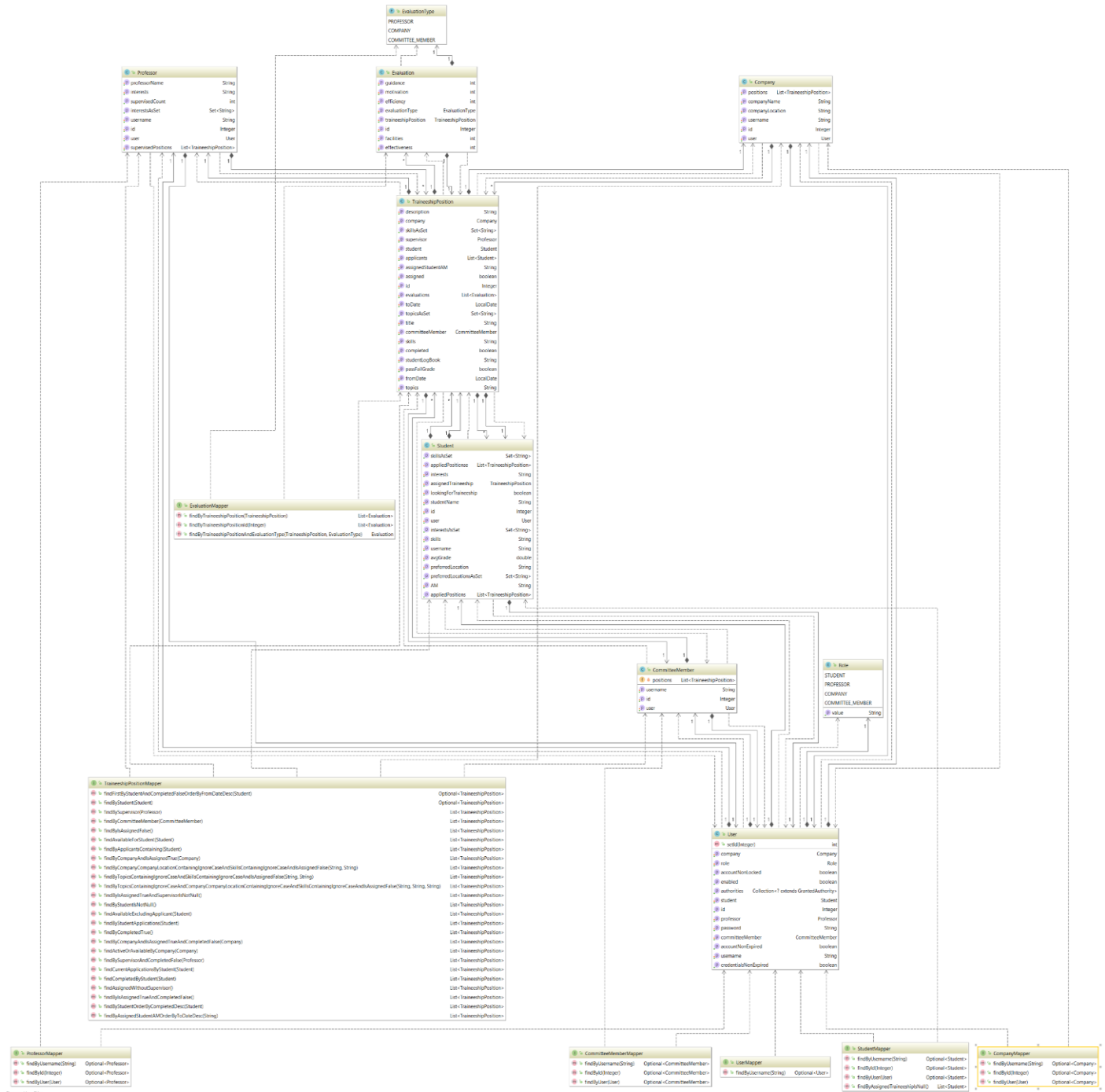


4.2 Design

Domain model:



Data Mappers:



Services:

StudentService		
findByUsername(String)	Student	
findById(Integer)	Student	
findByUser(User)	Student	
getCurrentStudent()	Student	
updateCurrentStudent(Student)	void	
applyForPosition(String, Integer)	void	
getAvailablePositionsForCurrentStudent()	List<TraineeshipPosition>	
getApplicationsForCurrentStudent()	List<TraineeshipPosition>	
appendToLogbook(String)	void	
getCompletedTraineeships()	List<TraineeshipPosition>	
findStudentsWithoutAssignedPosition()	List<Student>	

CommitteeMemberService		
getCurrentCommitteeMember()	CommitteeMember	
findByUsername(String)	CommitteeMember	
findByUser(User)	CommitteeMember	
findById(Integer)	CommitteeMember	
save(CommitteeMember)	CommitteeMember	
updateCurrentCommitteeMember(CommitteeMember)	void	
getFirstAvailable()	CommitteeMember	
findInProgressTraineeships()	List<TraineeshipPosition>	
findStudentsWithApplications(CommitteeMember)	List<Student>	
getSuggestedProfessors(Integer, String)	List<Professor>	

TraineeshipPositionService		
findOptionalByStudent(Student)	Optional<TraineeshipPosition>	
save(TraineeshipPosition)	void	
findById(Integer)	TraineeshipPosition	
deleteById(Integer)	void	
assignStudentToPosition(Integer, Integer)	void	
getAssignedPositionsByCompany(Company)	List<TraineeshipPosition>	
findAssignedWithoutSupervisor()	List<TraineeshipPosition>	
getAllCompleted()	List<TraineeshipPosition>	
findActiveByStudent(Student)	Optional<TraineeshipPosition>	

StudentServiceImpl		
studentMapper	StudentMapper	
userMapper	UserMapper	
traineeshipPositionMapper	TraineeshipPositionMapper	
findByUsername(String)	Student	
findById(Integer)	Student	
getCurrentStudent()	Student	
updateCurrentStudent(Student)	void	
findByUser(User)	Student	
applyForPosition(String, Integer)	void	
getAvailablePositionsForCurrentStudent()	List<TraineeshipPosition>	
appendToLogbook(String)	void	
getApplicationsForCurrentStudent()	List<TraineeshipPosition>	
findStudentsWithoutAssignedPosition()	List<Student>	
getCompletedTraineeships()	List<TraineeshipPosition>	

CommitteeMemberServiceImpl		
assignmentBasedOnLoad	AssignmentBasedOnLoad	
assignmentBasedOnInterests	AssignmentBasedOnInterests	
committeeMemberMapper	CommitteeMemberMapper	
userMapper	UserMapper	
traineeshipPositionMapper	TraineeshipPositionMapper	
getCurrentCommitteeMember()	CommitteeMember	
findByUsername(String)	CommitteeMember	
findByUser(User)	CommitteeMember	
findById(Integer)	CommitteeMember	
save(CommitteeMember)	CommitteeMember	
updateCurrentCommitteeMember(CommitteeMember)	void	
getFirstAvailable()	CommitteeMember	
getSuggestedProfessors(Integer, String)	List<Professor>	
findInProgressTraineeships()	List<TraineeshipPosition>	
findStudentsWithApplications(CommitteeMember)	List<Student>	

TraineeshipPositionServiceImpl		
traineeshipPositionMapper	TraineeshipPositionMapper	
studentMapper	StudentMapper	
findActiveByStudent(Student)	Optional<TraineeshipPosition>	
findOptionalByStudent(Student)	Optional<TraineeshipPosition>	
save(TraineeshipPosition)	void	
findById(Integer)	TraineeshipPosition	
deleteById(Integer)	void	
assignStudentToPosition(Integer, Integer)	void	
getAvailablePositionsByLocationAndSkills(String, String)	List<TraineeshipPosition>	
getAssignedPositionsByCompany(Company)	List<TraineeshipPosition>	
findAllAssignedPositions()	List<TraineeshipPosition>	
findAssignedWithoutSupervisor()	List<TraineeshipPosition>	
getAllCompleted()	List<TraineeshipPosition>	

CompanyService		
getCurrentCompany()	Company	
findByUsername(String)	Company	
findByUser(User)	Company	
save(Company)	Company	
updateCurrentCompany(Company)	void	
retrieveAvailablePositions()	List<TraineeshipPosition>	
createPositionForCurrentCompany(TraineeshipPosition, CommitteeMember)	void	

ProfessorService		
getCurrentProfessor()	Professor	
findByUsername(String)	Professor	
findByUser(User)	Professor	
updateCurrentProfessor(Professor)	void	
findById(Integer)	Professor	
getSupervisedPositions()	List<TraineeshipPosition>	

UserService		
saveUser(User)	void	
isUserPresent(User)	boolean	
findByUser(String)	User	
findByUsername(String)	User	

CompanyServiceImpl		
companyMapper	CompanyMapper	
userMapper	UserMapper	
positionMapper	TraineeshipPositionMapper	
getCurrentCompany()	Company	
createPositionForCurrentCompany(TraineeshipPosition, CommitteeMember)	void	
findByUsername(String)	Company	
findByUser(User)	Company	
save(Company)	Company	
updateCurrentCompany(Company)	void	
retrieveAvailablePositions()	List<TraineeshipPosition>	

ProfessorServiceImpl		
userMapper	UserMapper	
professorMapper	ProfessorMapper	
traineeshipPositionMapper	TraineeshipPositionMapper	
getCurrentProfessor()	Professor	
findByUsername(String)	Professor	
findByUser(User)	Professor	
updateCurrentProfessor(Professor)	void	
findById(Integer)	Professor	
getSupervisedPositions()	List<TraineeshipPosition>	

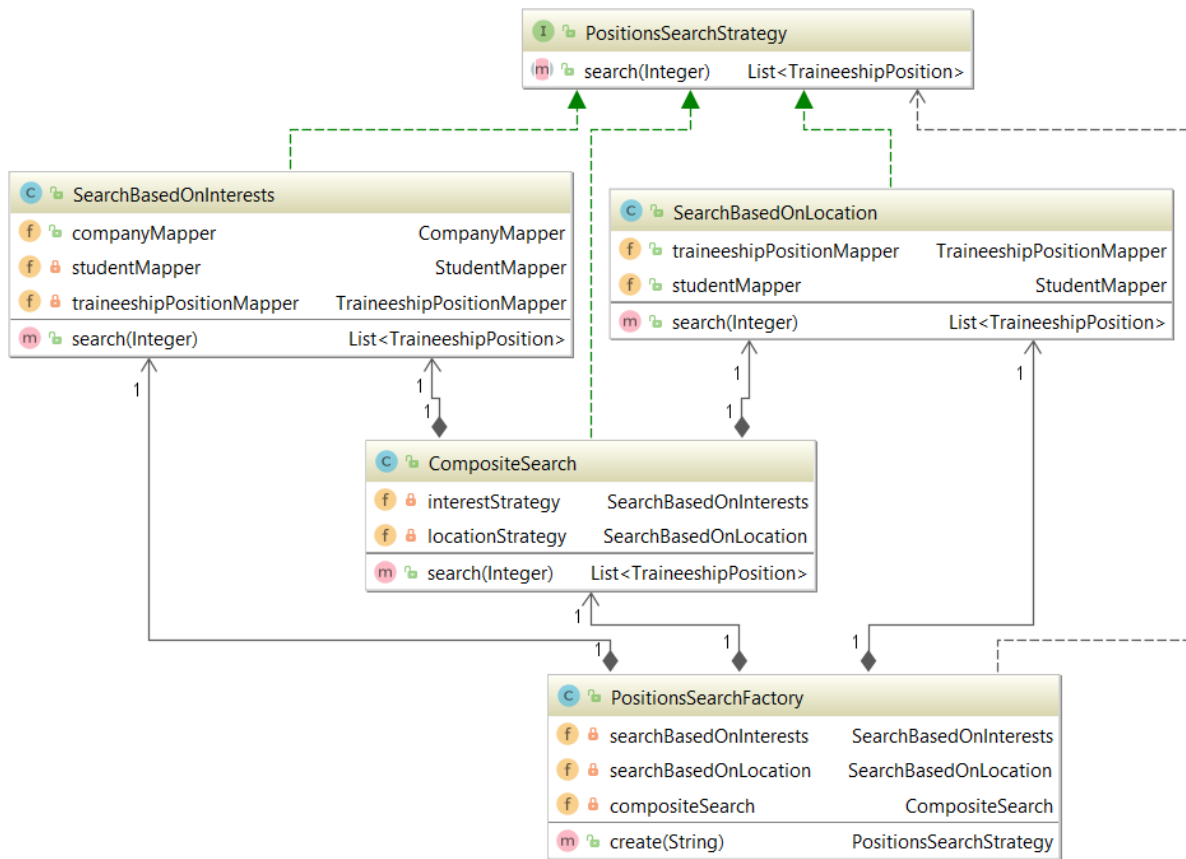
UserServiceImpl		
bCryptPasswordEncoder	BCryptPasswordEncoder	
userMapper	UserMapper	
saveUser(User)	void	
isUserPresent(User)	boolean	
loadUserByUsername(String)	UserDetails	
findById(String)	User	
findByUsername(String)	User	

EvaluationService		
findByTraineeshipPosition(TraineeshipPosition)	List<Evaluation>	
findByPositionAndType(TraineeshipPosition, EvaluationType)	Evaluation	
saveOrUpdateEvaluation(Evaluation)	void	

EvaluationServiceImpl		
evaluationMapper	EvaluationMapper	
traineeshipPositionMapper	TraineeshipPositionMapper	
findByTraineeshipPosition(TraineeshipPosition)	List<Evaluation>	
findByPositionAndType(TraineeshipPosition, EvaluationType)	Evaluation	
saveProfessorEvaluation(Evaluation)	void	
saveOrUpdateEvaluation(Evaluation)	void	
existsByPositionAndType(TraineeshipPosition, EvaluationType)	Evaluation	

Powered by yf-its

Search Strategy:

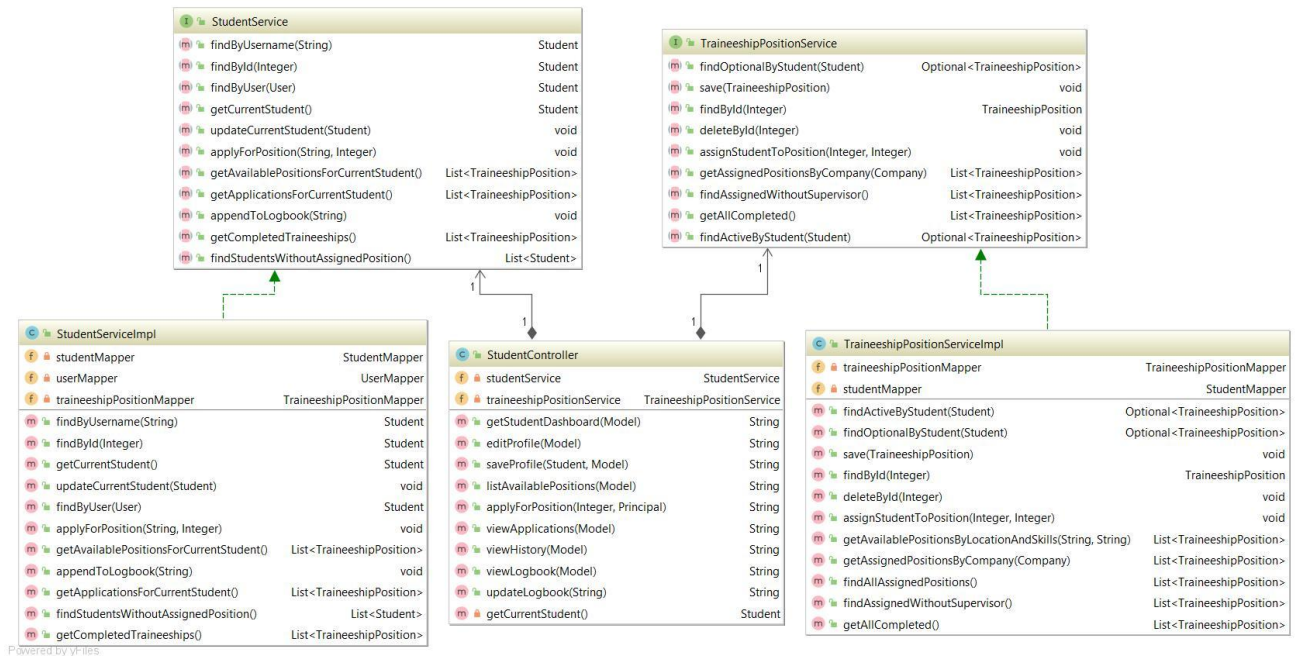


Powered by yFiles

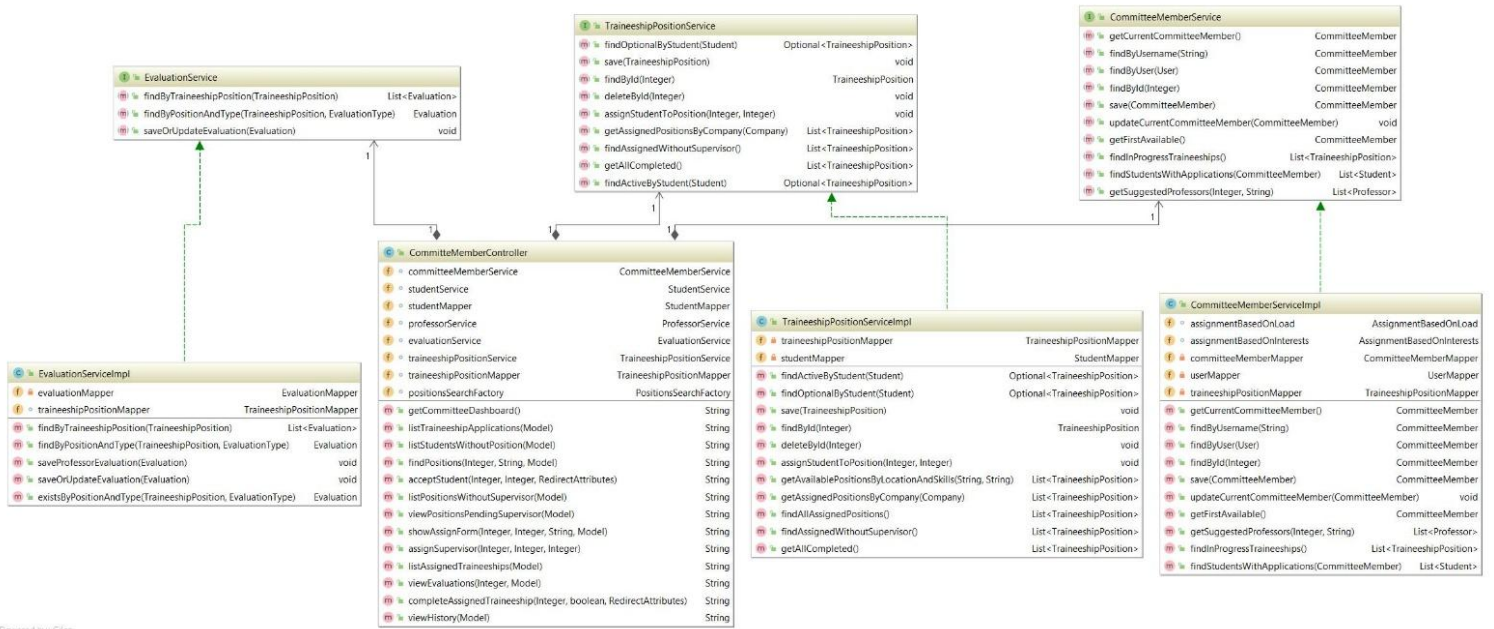
Controllers:



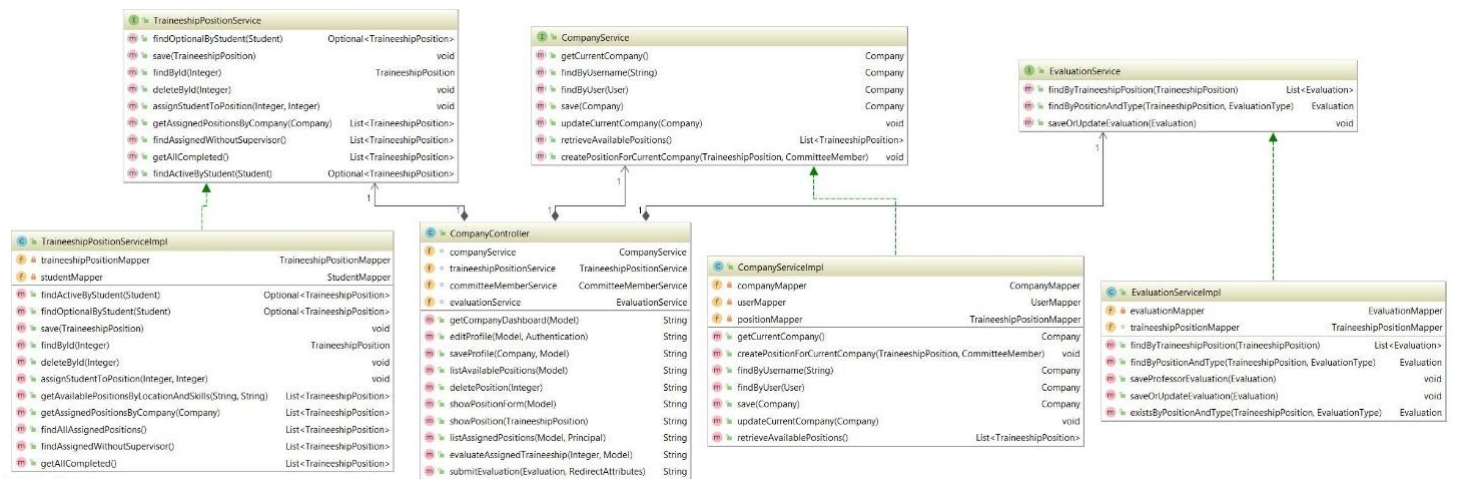
Student Controller:



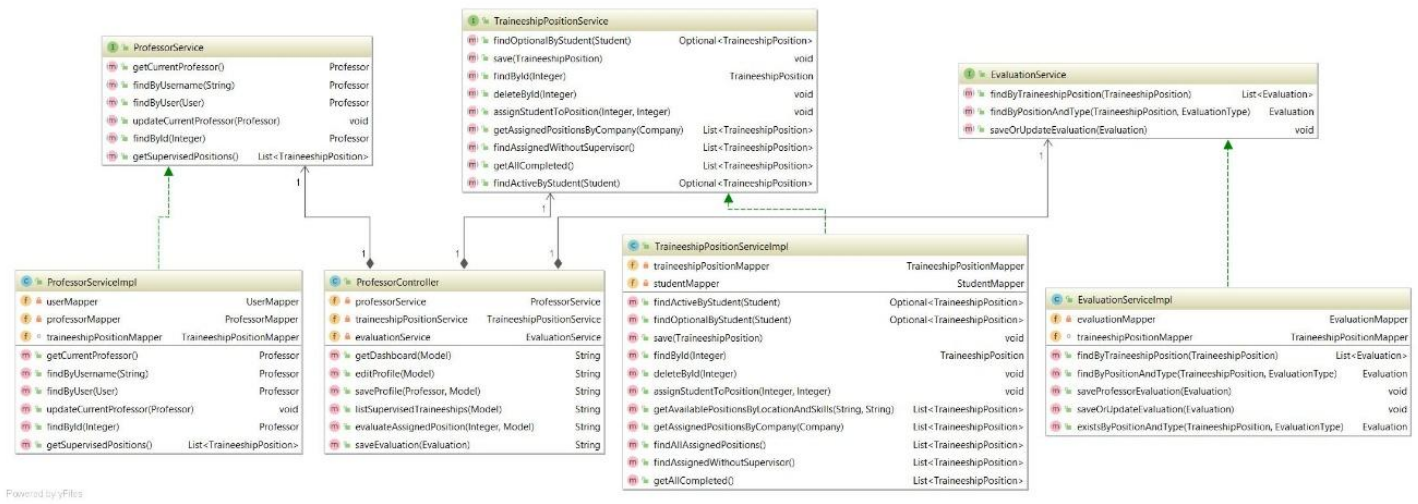
Committee Member Controller:



Company Controller:



Professor Controller:



The classes that are included in this release are given below:

Class Name: AuthController	
Responsibilities: Handles user login, registration, and redirection.	Collaborations: - Uses UserService. - Interacts with mappers: StudentMapper, ProfessorMapper, CompanyMapper, CommitteeMemberMapper.
Class Name: CommitteeMemberController	
Responsibilities: Committee user functionality: viewing applications, suggesting positions, assigning supervisors, and marking completions.	Collaborations: Uses CommitteeMemberService, ProfessorService, StudentService, TraineeshipPositionService, EvaluationService, PositionsSearchFactory.

Class Name: CompanyController	
Responsibilities: Allows company users to edit profiles, create and manage positions, and submit evaluations.	Collaborations: Interacts with CompanyService, CommitteeMemberService, TraineeshipPositionService, EvaluationService.

Class Name: ProfessorController	
Responsibilities: Handles professor dashboard, profile editing, supervision tracking, and evaluations.	Collaborations: Uses ProfessorService, EvaluationService, TraineeshipPositionService.

Class Name: StudentController	
Responsibilities: Manages student dashboard, profile, available positions, applications, and logbook updates.	Collaborations: Works with StudentService, TraineeshipPositionService.

Class Name: Student	
Responsibilities: Contains student profile data and preferences.	Collaborations: - One-to-one with User. - Many-to-many with TraineeshipPosition (applications). - One-to-one with assigned TraineeshipPosition.

Class Name: Professor	
Responsibilities: - Contains professor profile and areas of interest. - Supervises positions.	Collaborations: - One-to-one with User. - One-to-many with TraineeshipPosition as supervisor.

Class Name: Company	
Responsibilities: Represents companies offering traineeship positions.	Collaborations: - One-to-one with User. - One-to-many with TraineeshipPosition.

Class Name: CommitteeMember	
Responsibilities: Represents a committee member managing traineeship processes.	Collaborations: - One-to-one with User. - One-to-many with TraineeshipPosition.

Class Name: TraineeshipPosition	
Responsibilities: Describes a traineeship, including assignment and evaluation state.	Collaborations: - Linked to Company, Student, Professor, CommitteeMember, and Evaluation. - Many-to-many with Student (applicants).

Class Name: Evaluation	
Responsibilities: Captures evaluation scores and type (e.g. by professor or company).	Collaborations: - Belongs to a TraineeshipPosition. - Uses EvaluationType enum.

Class Name: User	
Responsibilities: - Represents application users. - Implements Spring Security's UserDetails.	Collaborations: One-to-one relationship with Student, Professor, Company, and CommitteeMember.

Class Name: Role	
Responsibilities: Enum for defining user roles: STUDENT, PROFESSOR, COMPANY, COMMITTEE_MEMBER.	Collaborations: Used in User for access control.

Class Name: EvaluationType	
Responsibilities: Represents evaluation types: PROFESSOR, COMPANY, COMMITTEE_MEMBER.	Collaborations: Used by Evaluation to define evaluation context.

Class Name: StudentMapper	
Responsibilities: Provides database access for Student entity.	Collaborations: Used in StudentServiceImpl for operations such as profile updates, application tracking, logbook updates.

Class Name: ProfessorMapper	
------------------------------------	--

Responsibilities: Manages database operations for Professor entity.	Collaborations: Used in ProfessorServiceImpl and assignment strategies to find suitable professors.
--	--

Class Name: CompanyMapper	
Responsibilities: Interfaces with the database for Company entities	Collaborations: Used in CompanyServiceImpl to retrieve and update company data.

Class Name: CommitteeMemberMapper	
Responsibilities: Manages persistence of CommitteeMember entities.	Collaborations: Used in CommitteeMemberServiceImpl for retrieving members and their responsibilities.

Class Name: TraineeshipPositionMapper	
Responsibilities: Main interface for CRUD and custom queries on TraineeshipPosition.	Collaborations: Extensively used across all services and strategies (matching, assignment, evaluation). Contains custom methods for filtering by status, company, student, supervisor, etc.

Class Name: UserMapper	
Responsibilities: - Διαχειρίζεται τους User χρήστες της εφαρμογής. - Υπεύθυνο για την authentication (βρίσκει χρήστες με βάση το username).	Collaborations: - Συνδέεται με User entity. - Χρησιμοποιείται από UserService και το authentication σύστημα της Spring Security.

Class Name: EvaluationMapper	
Responsibilities: Provides database access to Evaluation entities.	Collaborations: Used in EvaluationServiceImpl to fetch or save evaluations by position and type.

Class Name: StudentService	
Responsibilities: Provides student-related operations (profile update, application, logbook).	Collaborations: Used in StudentController.

Class Name: StudentServiceImpl	
Responsibilities: Implements student operations.	Collaborations: Works with Student, TraineeshipPosition, StudentMapper.

Class Name: ProfessorService	
Responsibilities: Handles professor-specific features.	Collaborations: Interacts with TraineeshipPosition, Evaluation, Professor.

Class Name: ProfessorServiceImpl	
Responsibilities: Implements ProfessorService.	Collaborations: Works with ProfessorMapper, EvaluationService, TraineeshipPositionMapper.

Class Name: CompanyService	
Responsibilities: Manages company operations.	Collaborations: Used in CompanyController.

Class Name: CompanyServiceImpl	
Responsibilities: Implements CommitteeMemberService logic.	Collaborations: Uses StudentMapper, ProfessorMapper, TraineeshipPositionMapper, etc.

Class Name: CommitteeMemberService	
Responsibilities: Provides committee-related operations (student applications, professor assignment).	Collaborations: Used in CommitteMemberController.

Class Name: CommitteeMemberServiceImpl	
Responsibilities: Implements CommitteeMemberService logic.	Collaborations: Uses StudentMapper, ProfessorMapper, TraineeshipPositionMapper, etc.

Class Name: TraineeshipPositionService	
Responsibilities: Handles traineeship operations like create, assign, evaluate.	Collaborations: Used across multiple controllers.

Class Name: TraineeshipPositionServiceImpl	
Responsibilities: ▪ Implements core logic for managing TraineeshipPosition entities.	Collaborations: ▪ Uses multiple mappers and services.

Class Name: EvaluationService	
Responsibilities: ▪ Handles logic for creating and retrieving evaluations.	Collaborations: ▪ Used in CompanyController, ProfessorController.

Class Name: EvaluationServiceImpl	
Responsibilities: ▪ Implements evaluation operations.	Collaborations: ▪ Works with EvaluationMapper, TraineeshipPositionService.

Class Name: UserService	
Responsibilities: ▪ Handles user operations like saving and checking for duplicates.	Collaborations: ▪ Used by AuthController.

Class Name: UserServiceImpl	
Responsibilities: ▪ Implements UserService logic.	Collaborations: ▪ Accesses user data, integrates with Spring Security.

Class Name: WebSecurityConfig	
Responsibilities:	Collaborations: ▪ Uses CustomSecuritySuccessHandler.

▪ Configures application security rules (role-based access, login/logout behavior). ▪ Registers the security filter chain.	▪ Injects UserServiceImpl as UserDetailsService.
---	--

Class Name: CustomSecuritySuccessHandler	
Responsibilities: ▪ Handles redirection after successful login based on user role.	Collaborations: ▪ Works with Spring Security login process. ▪ Uses User and its roles to determine redirection URL.

Class Name: WebMvcConfig	
Responsibilities: ▪ Registers simple view controller for root (e.g., / → homepage).	Collaborations: ▪ Used by Spring MVC configuration.

Class Name: SupervisorAssignmentFactory	
Responsibilities: ▪ Returns an assignment strategy (e.g., based on interests or load).	Collaborations: ▪ Returns AssignmentBasedOnLoad or AssignmentBasedOnInterests.

Class Name: AssignmentBasedOnLoad	
Responsibilities: ▪ Assigns the least busy professor to a traineeship.	Collaborations: ▪ Uses ProfessorMapper and TraineeshipPositionMapper.

Class Name: AssignmentBasedOnInterests	
Responsibilities: ▪ Assigns a professor whose interests best match the traineeship topics.	Collaborations: ▪ Uses ProfessorMapper and TraineeshipPositionMapper.

Class Name: SupervisorAssignmentStrategy	
Responsibilities: Interface for professor assignment strategies.	Collaborations: Implemented by both AssignmentBasedOnLoad and AssignmentBasedOnInterests.

Class Name: PositionsSearchFactory	
Responsibilities: Chooses the correct search strategy based on a given key (e.g., "interests", "location").	Collaborations: Returns instances of SearchBasedOnInterests, SearchBasedOnLocation, or CompositeSearch.

Class Name: SearchBasedOnInterests	
Responsibilities: Finds traineeships matching the student's interests.	Collaborations: Uses StudentMapper, TraineeshipPositionMapper.

Class Name: SearchBasedOnLocation	
Responsibilities: Finds traineeships based on the student's preferred locations.	Collaborations: Uses StudentMapper, TraineeshipPositionMapper.

Class Name: CompositeSearch	
Responsibilities: Combines results from interest-based and location-based search strategies.	Collaborations: Uses both SearchBasedOnInterests and SearchBasedOnLocation.

Class Name: PositionsSearchStrategy	
Responsibilities:	Collaborations:

▪ Interface for all position search strategies.	▪ Implemented by all search strategy classes.
---	---

.....

.....